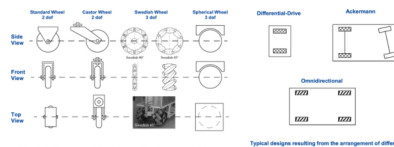


Wheel Types and Configurations



Stability, Maneuverability and Controllability

- Stability**
 - Depends on center of mass and the polygon spanned by wheel contact points
 - Center of mass has to be within area of the polygon
- Maneuverability**
 - Describes the range/flexibility of motions of a mobile robot
 - High maneuverability: Robot can move in many possible directions or change motion quickly (e.g., translate and rotate in place)
- Controllability**
 - Inverse relationship: Higher maneuverability → lower controllability
 - More motions and degrees of freedom require more complex control

Configuration Space (C-Space) and Degrees of Freedom

- The mathematical space containing all possible states a robot can be in
- Defined by the smallest number of variables needed to perfectly represent the robot's current state
- Examples**
 - Manipulators: The values of its joints (e.g., joint angles)
 - Mobile Robots: Position (x,y) and orientation (θ) relative to a reference frame
- The number of dimensions in a robot's C-space exactly equals its Degrees of Freedom (DOF)
- C-Space Representations:** Continuous, Graphs, & Grids

Robot Poses and Transformation Matrices

- We often use transformation matrices to express a robot's pose (position and orientation)
- Transformation matrices combine position and orientation
- Commonly used to express the pose of a frame w.r.t. another frame

Position (Translation) $T_{10} = \begin{bmatrix} R_{10} & p_1 \\ 0 & 1 \end{bmatrix}$

Orientation (Rotation) $T_{10} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Mobile Robot Velocities

- Like poses, velocities can be expressed in different frames
- The velocity of the robot (frame B) expressed in the global frame I is $\dot{p}_B = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}$
- The velocity of the robot (frame B) expressed in the robot frame B is $\dot{p}_B = \begin{bmatrix} \dot{x}_B \\ \dot{y}_B \end{bmatrix}$

Differential Kinematics

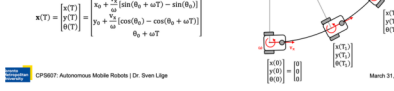
- The differential kinematics relate a robot's wheel velocities (control inputs) to its resulting body-centric velocities
- Inverse differential kinematics for differential-drive robots $\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{1}{r} \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix}$
- Forward differential kinematics for differential-drive robots $\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \frac{r}{\cos(\theta)} \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}$

Mobile Robot Motion Model

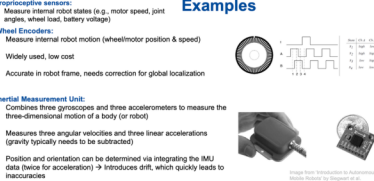
- The motion model relates the robot's control inputs (usually velocities) to the robot's global velocity $\dot{x} = A(x)\dot{u} + B(x)u$
- For our differential drive example, we have $\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \omega \end{bmatrix}$
- Sometimes, the body-centric velocities are directly considered as inputs $\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \omega \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}$

Differential Drive: Unicycle Model

- For differential drive robots, we have the classic unicycle model $\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$
- If we assume constant velocities v_x and ω over time interval T and zero initial conditions, we can integrate this expression in discrete form



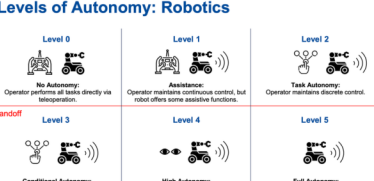
Robot Motion Planning Definitions



Active vs. Passive Sensing

- Exteroceptive Sensors: Examples**
 - Stereo Camera: A stereo camera features two monocular cameras in a known configuration
 - Laser RangeFinder (LiDAR): Pulsed laser range-finders determine range by measuring the time of flight
 - Active sensors: Measure external environmental energy (e.g., temperature sensors, microphones, cameras)
- Properties of Path Planners**
 - Completeness: Complete: A complete planner either finds a feasible path or reports that there is none in finite time
 - Optimality: Optimal: An optimal planner always finds the best (i.e., lowest cost) path, if there exists one

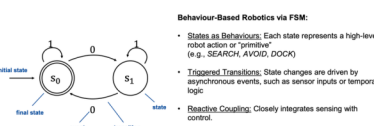
Levels of Autonomy: Robotics



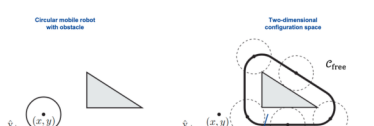
Autonomy Through Behaviors and Braitenberg Vehicles

- We learned that autonomy could emerge through simple rules and behaviors
- One example of the simple Braitenberg vehicles
- Goal: Model the animal world in a minimalist and constructive way, from simple reactive behaviors

Autonomy via Finite State Machines in Robotics



Free and Obstructed Configuration Space



Wheeled Mobile Robot Design and Kinematics



Sensors and Autonomy through Behaviors



State Estimation



Properties of Path Planners

- Completeness**
 - Complete: A complete planner either finds a feasible path or reports that there is none in finite time
 - Optimality: Optimal: An optimal planner always finds the best (i.e., lowest cost) path, if there exists one

Limitations of Open-Loop Control

- In practice, the robot rarely moves exactly as predicted by the model
- Common sources of error include wheel slip, uneven terrain, actuator imperfections or unmodeled dynamic effects
- Small motion errors accumulate over time and distance

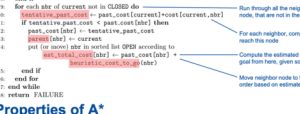
Open-Loop vs. Closed-Loop Control



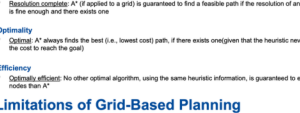
PID-Controller

Term	Robot	Effect on Rise Time	Effect on Overshoot	Effect on Steady-State Error
Proportional (P)	Drives toward reference	Decreases	Increases	Decreases
Integral (I)	Eliminates steady state	Increases	Increases	Eliminates
Derivative (D)	Dampens oscillation	Decreases	Decreases	No effect

Path-Tracking: Error Definitions



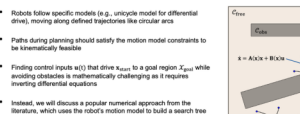
Closed-Loop Path-Tracking Controller



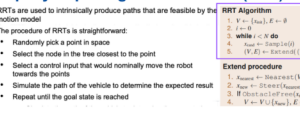
Rapidly-Exploring Random Trees (RRT)

- RRTs are used to intrinsically produce paths that are feasible by the motion model
- The procedure of RRTs is straightforward:
 - Randomly pick a point in space
 - Select the node in the tree closest to the point
 - Select a control input that would normally move the robot towards the point
 - Simulate the path of the vehicle to determine the expected result
 - Repeat until the goal state is reached

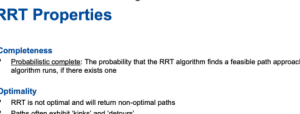
Planning with Vehicle Models



Correction Step Example



Typical Factors in a Factor Graph



Example Full Autonomy Stack



RRT* Algorithm

- To address the limitations regarding optimality of RRT, RRT* was proposed
- Two modifications w.r.t. conventional RRTs are made
- The algorithm is not terminated when the goal is reached
 - A second termination criteria is defined (e.g., maximum number of iterations, maximum tree size, maximum runtime)
 - When finished, the path with the shortest distance will be returned
 - Solution can improve as time goes by

Properties of RRT*

- Completeness**
 - Probabilistic completeness: The probability that the RRT* algorithm finds a feasible path approaches one, the longer the algorithm runs, if there exists one

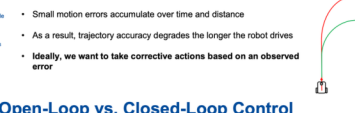
Naive Open-Loop Control

- Assume we want to follow a path composed of straight segments and constant curvature arcs
- We can first compute a time sequence of body-centric velocities that would allow the robot to follow this path
- We can then transform these body-centric velocities into wheel velocities control inputs to drive the robot on the path

Limitations of Open-Loop Control

- In practice, the robot rarely moves exactly as predicted by the model
- Common sources of error include wheel slip, uneven terrain, actuator imperfections or unmodeled dynamic effects
- Small motion errors accumulate over time and distance

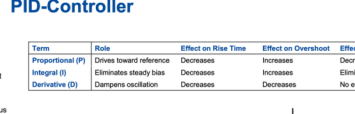
Open-Loop vs. Closed-Loop Control



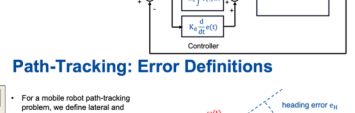
PID-Controller

Term	Robot	Effect on Rise Time	Effect on Overshoot	Effect on Steady-State Error
Proportional (P)	Drives toward reference	Decreases	Increases	Decreases
Integral (I)	Eliminates steady state	Increases	Increases	Eliminates
Derivative (D)	Dampens oscillation	Decreases	Decreases	No effect

Path-Tracking: Error Definitions



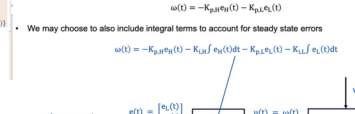
Closed-Loop Path-Tracking Controller



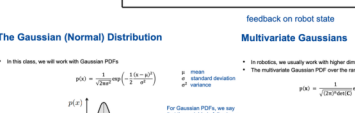
Rapidly-Exploring Random Trees (RRT)

- RRTs are used to intrinsically produce paths that are feasible by the motion model
- The procedure of RRTs is straightforward:
 - Randomly pick a point in space
 - Select the node in the tree closest to the point
 - Select a control input that would normally move the robot towards the point
 - Simulate the path of the vehicle to determine the expected result
 - Repeat until the goal state is reached

Planning with Vehicle Models



Correction Step Example



Typical Factors in a Factor Graph



Example Full Autonomy Stack



Noisy Wheel Velocities

$\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix}, C_\phi\right)$

$C_\phi = \begin{bmatrix} \sigma_\phi^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix}$

$x(t_{k+1}) \sim \mathcal{N}(g(\bar{x}(t_k), \bar{u}), G_x C_{x_k} G_x^T + G_u C_u G_u^T)$

$x(t_{k+1}) \sim \mathcal{N}(g(\bar{x}(t_k), \bar{u}), GCG^T + R)$

Joint covariance matrix for wheel velocities

$C_\phi = \begin{bmatrix} \sigma_\phi^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix}$

Uncertainty Ellipsoid Examples



Smoothing (Batch Estimation)

- Instead of updating only the current pose, we solve for the entire trajectory simultaneously
- We incorporate every available piece of information into a single system, including control inputs via the motion model and measurements
- The problem is usually formulated as a mathematical optimization where odometry and measurement information act as probabilistic constraints

Filtering vs. Smoothing

- Filtering**
 - Estimates the state $x(t_k)$ given all previous measurements $\{x(t_0), \dots, x(t_{k-1})\}$ and controls $\{u(t_0), \dots, u(t_{k-1})\}$
 - Once a measurement is used to update the state, the measurement and the previous state are effectively discarded
 - Filtering is fast and memory-efficient
- Smoothing**
 - Estimates the state $x(t_k)$ given all measurements $\{x(t_0), \dots, x(t_k)\}$ and controls $\{u(t_0), \dots, u(t_k)\}$ along the trajectory
 - Uses all available data to find the most likely "batch" the robot took
 - New information at time t_k can be used to improve the estimate of where the robot was at past times $t_0, \dots, t_{k-1}$
 - This allows us to obtain globally consistent estimates
 - Smoothing is more computationally intensive, but more accurate

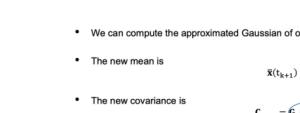
Factor Graphs for Smoothing

- Batch estimation problems are often visually represented by factor graphs
- Nodes represent the variables we wish to estimate
- Factors represent our probabilistic constraints (our available information)
- Edges represent dependencies and connect factors to the variables they depend on

Extended Kalman Filter: The Prediction Step

- We can compute the approximated Gaussian of our new state
- The new mean is $\bar{x}(t_{k+1}) = g(\bar{x}(t_k), \bar{u})$
- The new covariance is $C_{x_{k+1}} = G_x C_{x_k} G_x^T + G_u C_u G_u^T$
- The total expression is $\bar{x}(t_{k+1}) \sim \mathcal{N}(g(\bar{x}(t_k), \bar{u}), G_x C_{x_k} G_x^T + G_u C_u G_u^T)$

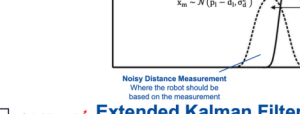
Correction Step Example



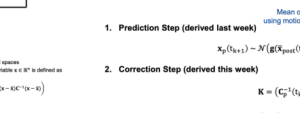
Typical Factors in a Factor Graph



Example Full Autonomy Stack



Path-Tracking Control



State Estimation



Localization-Only vs. SLAM



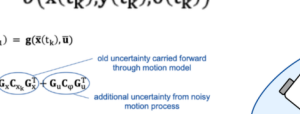
Simultaneous Localization and Mapping (SLAM)

- Simultaneous Localization and Mapping (SLAM)** is the process by which a robot builds a map of an unknown environment while simultaneously using that map to determine its own location
- The **Chicken-and-Egg Problem**
 - To **localize**, the robot needs a consistent map to identify landmarks and correct its position
 - To **map**, the robot needs an accurate estimate of its own location to correctly place sensor observations into a global frame

Mutual Dependency

- A better pose estimate leads to a more accurate map
- A more accurate map leads to better localization

Localization-Only vs. SLAM

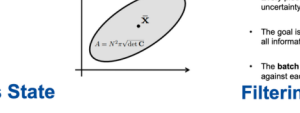


The Inverse Covariance (Information) Matrix

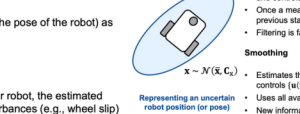
$\Sigma^{-1}(x) = \Sigma^{-1}(x_k) + \Sigma^{-1}(x_{k+1})$

$\Sigma^{-1}(x) = \Sigma^{-1}(x_k) + \Sigma^{-1}(x_{k+1})$

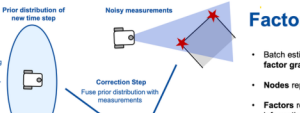
The Information Matrix and Correlations



Factor Graphs for Smoothing



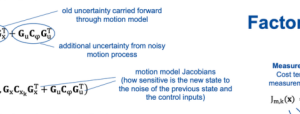
Exploiting Sparsity



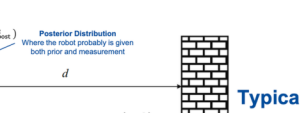
Solving the Resulting Optimization Problem

- For SLAM, we have the following combined state we seek to estimate $x = [x(t_0), \dots, x(t_k), p_1, \dots, p_N]$
- It includes both the robot states at discrete times $x(t_k)$ (k in total) and the position of each landmark p_i (N in total)
- Given all measurement and prior factors, we seek to find the minimum of the following optimization problem

Example Full Autonomy Stack



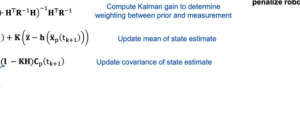
Path-Tracking Control



State Estimation



Localization-Only vs. SLAM



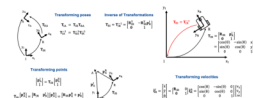
The Gaussian (Normal) Distribution



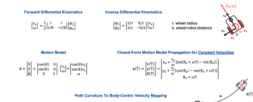
Multivariate Gaussians



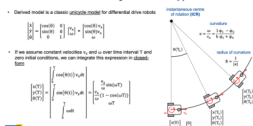
Cheat Sheet: Transformations



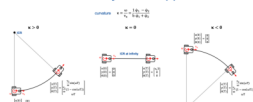
Cheat Sheet: Differential Drive Kinematics



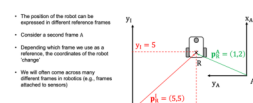
Differential Drive: Unicycle Model (I)



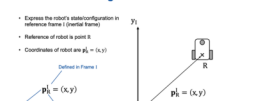
Differential Drive: Unicycle Model (II)



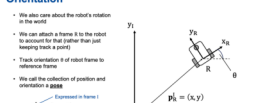
More on Frames



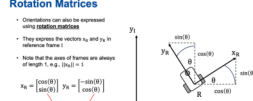
Mobile Robot Configuration



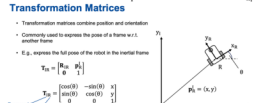
Orientation



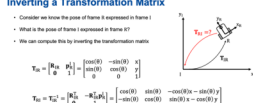
Rotation Matrices



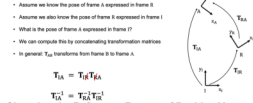
Transformation Matrices



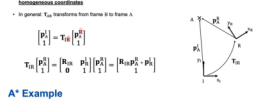
Inverting a Transformation Matrix



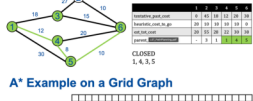
Changing the Reference Frame



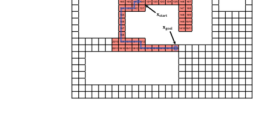
Changing the Reference Frames of Position Vectors



A* Example



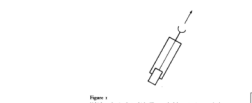
A* Example on a Grid Graph



Levels of Autonomy: Self Driving



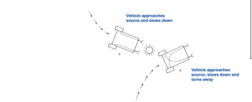
Vehicle 1: Getting Around



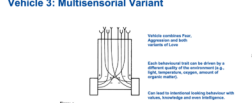
Vehicle 2: Fear and Aggression



Vehicle 3: Love



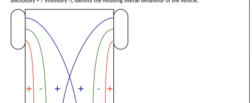
Vehicle 3: Multisensorial Variant



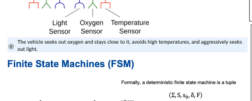
Vehicle 4: Values and Special Tastes



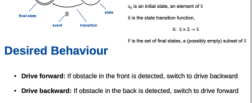
Finite State Machines (FSM)



Desired Behavior



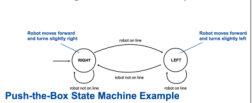
Finite State Machine Diagram



Line-Following State Machine Example



Push-the-Box State Machine Example



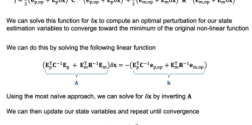
Gauss-Newton Optimization I



Gauss-Newton Optimization II



Pulse-Width Modulation



Joint and Wheel Constraints



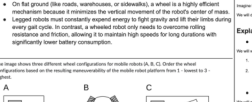
Using the Motion Model as the Prior



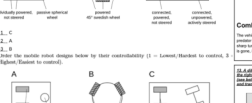
Controllability



Maneuverability



RRT* Example



Using PDFs to Compute Probability



Predicting Future States: Wheel Odometry



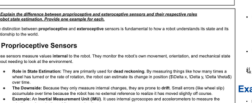
Summary of Results



Two Wheel vs. Four Wheel Robot



Correcting our Motion Model Estimate with Sensing



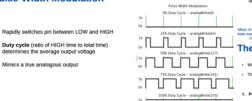
Stability



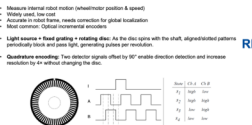
Maneuverability



Controllability



Maneuverability



Controllability



Maneuverability



Transformations and Mobile Robot Kinematics [25 points]



Component List & Connection Logic



Wiring Diagram & Component List



Explanation of Behaviors



Combined Design Behavior



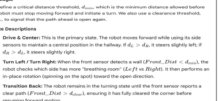
Finite State Machine Design



Mean or Value Estimation (E.g.)



Summary of Results



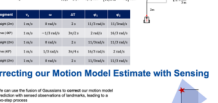
Two Wheel vs. Four Wheel Robot



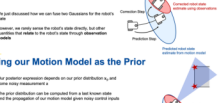
Correcting our Motion Model Estimate with Sensing



Stability



Maneuverability



Controllability



Maneuverability



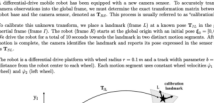
Controllability



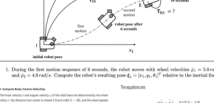
Maneuverability



Robot Localization and Mapping [25 points]



Summary of Results



Two Wheel vs. Four Wheel Robot



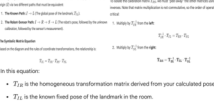
Correcting our Motion Model Estimate with Sensing



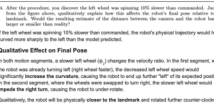
Stability



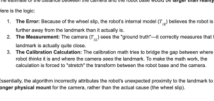
Maneuverability



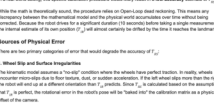
Controllability



Maneuverability



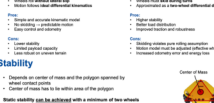
Controllability



Maneuverability



Controllability



Maneuverability



Controllability



Maneuverability



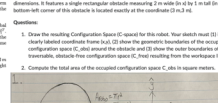
Controllability



Maneuverability



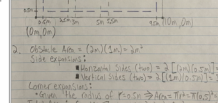
Robot Localization and Mapping [25 points]



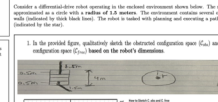
Summary of Results



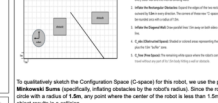
Two Wheel vs. Four Wheel Robot



Correcting our Motion Model Estimate with Sensing



Stability



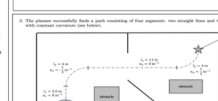
Maneuverability



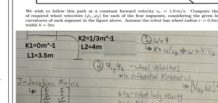
Controllability



Maneuverability



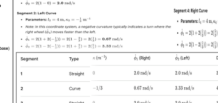
Controllability



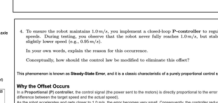
Maneuverability



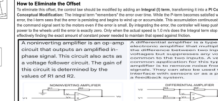
Controllability



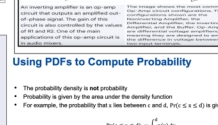
Maneuverability



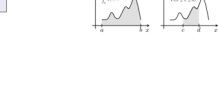
Controllability



Maneuverability



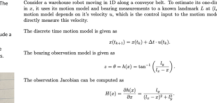
Controllability



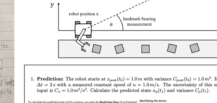
Maneuverability



Robot Localization and Mapping [25 points]



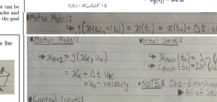
Summary of Results



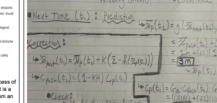
Two Wheel vs. Four Wheel Robot



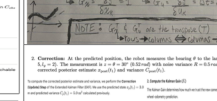
Correcting our Motion Model Estimate with Sensing



Stability



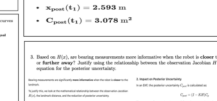
Maneuverability



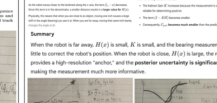
Controllability



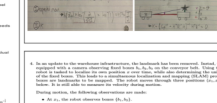
Maneuverability



Controllability



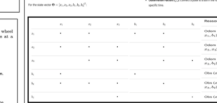
Maneuverability



Controllability



Maneuverability



Controllability



Maneuverability



Controllability



Maneuverability

